

18.1. Excepciones

Una **excepción** es un tipo especial de error que ocurre mientras el programa se está ejecutando.

Por ejemplo:

- Dividir por cero.
- Convertir un texto a número y que no sea válido.
- Acceder a un archivo que no existe.

Estas situaciones provocan errores **predecibles**, y con **try/except** podemos **anticiparnos** y actuar de forma segura.

```
try:
    # Código que podría fallar
except TipoDeError:
    # Código que se ejecuta si ocurre ese error
```

- El bloque **try** contiene el código que puede lanzar una excepción.
- Si ocurre una excepción, Python **salta al bloque except** correspondiente.
- Si no ocurre ningún error, el bloque **except** no se ejecuta.

También se puede usar un **except** general sin especificar el tipo de error, aunque no es lo más recomendado:

```
try:
    # Código que puede fallar
except:
    print("Ocurrió un error")
```

El uso de **try** y **except** permite que tus programas:

- No se detengan por errores controlables.
- Den mensajes amigables al usuario.
- Sean más robustos y confiables.

18.1.1. Tipos de excepciones comunes en Python (algunos de ellos)

En Python existen muchos tipos de excepciones, pero para un curso introductorio alcanza con conocer algunas de las más frecuentes. A continuación se explican las más comunes, acompañadas de ejemplos para facilitar su comprensión.



ZeroDivisionError

Se produce cuando intentamos dividir un número por cero.

```
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("No se puede dividir por cero.")
```

El ejemplo es literal pero si fueran valores ingresados por un usuario y nos encontramos que el divisor es ingresado como 0 captamos el error para que el programa no se detenga, sino que lance el error y siga su ejecución.

ValueError

Ocurre cuando intentamos convertir datos a un tipo incompatible. Por ejemplo, convertir letras a número.

```
try:
    numero = int("hola")
except ValueError:
    print("Eso no es un número válido.")
```

FileNotFoundError

Aparece cuando se intenta abrir un archivo que no existe.

```
try:
    archivo = open("inexistente.txt", "r")
except FileNotFoundError:
    print("El archivo no se encontró.")
```

KeyError

Se lanza cuando tratamos de acceder a una clave que no existe en un diccionario.

```
datos = {"nombre": "Ana"}

try:
    print(datos["edad"])
except KeyError:
    print("Esa clave no existe en el diccionario.")
```

IndexError

Se genera al intentar acceder a una posición que no existe en una lista o tupla.

```
lista = [10, 20, 30]

try:
    print(lista[5])
except IndexError:
    print("Índice fuera de rango.")
```

TypeError

Se lanza cuando se realizan operaciones entre tipos de datos incompatibles.

```
try:
    resultado = "5" + 2
except TypeError:
    print("No se pueden sumar cadenas y números.")
```